

# Module 1: Python essentials for health professionals

Data analysis with Python for health specialists

---

Yaé Ulrich Gaba

2026

AIRINA Labs

# Why are we here?

You are health professionals. You already know:

- How to interpret lab values
- What a p-value means (roughly)
- That Excel has limits

Today you learn **one new tool** that makes your existing skills more powerful.



Clinical knowledge  
+ Python

= reproducible research

# Why Python, not Excel?

|                   | Excel             | Python              |
|-------------------|-------------------|---------------------|
| Reproducibility   | ✗ Click-based     | ✓ Script-based      |
| Scale             | Slow at 100K rows | Handles millions    |
| Survival analysis | No                | <i>lifelines</i>    |
| Disease mapping   | No                | <i>geopandas</i>    |
| ML prediction     | No                | <i>scikit-learn</i> |
| Cost              | \$150/yr          | Free                |

## Your workspace: Jupyter notebooks

Code cell: `print("Hello, health data!")`

Output: Hello, health data!

Markdown cell: Formatted text, headings, notes

 Google Colab: [colab.research.google.com](https://colab.research.google.com) — no installation needed

 Local: Anaconda → Jupyter Notebook

## Variables: storing patient data

```
patient_age = 45           # int
temperature = 37.2         # float
diagnosis = "Type 2 Diabetes" # str
is_hospitalized = True     # bool
```

### Naming convention

Use descriptive names: *patient\_age*, not *x*. Your future self will thank you.

## Collections: lists and dictionaries

List — ordered values:

```
bp = [120, 135, 128, 142]
print(bp[0])    # 120
print(len(bp)) # 4
```

Dictionary — key-value pairs:

```
patient = {
    "id": "PAT-42",
    "age": 58,
    "dx": "HTN",
}
print(patient["dx"])
```

Think of a dictionary as a **patient chart** in code.

## Control flow: clinical classification

```
systolic = 148

if systolic >= 140:
    category = "Stage 2 Hypertension"
elif systolic >= 130:
    category = "Stage 1 Hypertension"
elif systolic >= 120:
    category = "Elevated"
else:
    category = "Normal"
```

Clinical mapping: AHA/ACC 2017 guidelines → Python code.

## Loops: screening multiple patients

```
patients = [  
    {"id": "P001", "hba1c": 5.4},  
    {"id": "P002", "hba1c": 7.8},  
    {"id": "P003", "hba1c": 6.1},  
]  
  
for p in patients:  
    if p["hba1c"] >= 6.5:  
        print(f"{p['id']}: Diabetic")  
    elif p["hba1c"] >= 5.7:  
        print(f"{p['id']}: Pre-diabetic")  
    else:  
        print(f"{p['id']}: Normal")
```



## Functions: reusable calculations

```
def bmi(weight_kg, height_m):  
    """Calculate Body Mass Index."""  
    return weight_kg / (height_m ** 2)  
  
def bmi_category(value):  
    if value < 18.5: return "Underweight"  
    elif value < 25: return "Normal"  
    elif value < 30: return "Overweight"  
    else: return "Obese"  
  
print(bmi(82, 1.75))  # 26.8
```

Write once, use everywhere. Functions = reproducibility.

## Importing libraries

```
import pandas as pd          # data tables
import numpy as np           # numbers
import matplotlib.pyplot as plt # plots
import seaborn as sns        # statistical plots
from scipy import stats      # tests
```

These five lines start **every** health data analysis notebook.

Install if missing

```
pip install pandas matplotlib seaborn scipy
```

```
In Colab: !pip install lifelines
```

## What you can do after this module

1. Store patient data in Python variables, lists, and dictionaries
2. Classify clinical measurements with *if/elif/else*
3. Write reusable functions (BMI, eGFR, BP category)
4. Loop through patient records for batch screening
5. Import the core libraries for the rest of this course

Next: Module 2 — Health data with Pandas

## Exercises (Hour 3)

1. **BP classifier:** Write `classify_bp(systolic, diastolic)` returning AHA category
2. **BMI batch:** Create 5 patient dicts with weight/height, compute and print BMI for each
3. **eGFR function:** Implement the CKD-EPI equation
4. **Mini-project:** Build a patient intake tool that prints a formatted clinical summary